

STAGE 3: Flutter Basics (Widgets, Layouts)

Flutter & Dart: From Zero to Expert — Stage Textbook 3 of 8

Goal: Understand how Flutter apps are built from widgets, and be able to lay out real screens.

Estimated time: 3-4 weeks

Prerequisite: Stage 2 (Dart Intermediate)

3.1 How Flutter Works: Everything is a Widget

Flutter builds UI by composing **widgets** into a tree. Each widget describes part of the screen, and Flutter re-renders only the parts of the tree that change.

Install Flutter: <https://docs.flutter.dev/get-started/install> Create a new project: `flutter create my_app` Run it: `flutter run`

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Hello Flutter')),
        body: const Center(child: Text('Welcome!')),
      ),
    );
  }
}
```

3.2 StatelessWidget vs StatefulWidget

```

// Stateless: renders once based on input, never changes itself
class Greeting extends StatelessWidget {
  final String name;
  const Greeting({super.key, required this.name});

  @override
  Widget build(BuildContext context) => Text('Hello, $name!');
}

// Stateful: can change its own appearance over time
class Counter extends StatefulWidget {
  const Counter({super.key});

  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int _count = 0;

  void _increment() {
    setState(() => _count++);
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Count: $_count'),
        ElevatedButton(onPressed: _increment, child: const Text('+1')),
      ],
    );
  }
}

```

Key rules:

- Use `StatelessWidget` when the widget never needs to redraw itself based on internal data.
- Use `StatefulWidget` + `setState()` whenever the widget owns data that changes (counters, form fields, toggles).
- `setState()` is what tells Flutter "rebuild this widget and its children."

3.3 Core Layout Widgets

```

Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    const Text('Title', style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
    const SizedBox(height: 8),
    Row(
      children: [
        const Icon(Icons.star, color: Colors.amber),
        const SizedBox(width: 4),
        const Text('4.8 rating'),
      ],
    ),
    Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Colors.blue.shade50,
        borderRadius: BorderRadius.circular(8),
      ),
      child: const Text('A styled card-like box'),
    ),
    Expanded(
      child: Stack(
        children: [
          Container(color: Colors.grey.shade300),
          const Positioned(bottom: 8, right: 8, child: Text('Overlay text')),
        ],
      ),
    ),
  ],
)

```

Key rules:

- `Column` stacks children vertically, `Row` stacks them horizontally.
- `Container` is your general-purpose box for padding, margin, color, borders, and decoration.
- `Expanded` makes a child fill remaining space inside a `Row` / `Column`.
- `Stack` + `Positioned` lets you overlay widgets on top of each other.

3.4 Input Widgets

```

class LoginForm extends StatefulWidget {
  const LoginForm({super.key});
  @override
  State<LoginForm> createState() => _LoginFormState();
}

class _LoginFormState extends State<LoginForm> {
  final _controller = TextEditingController();
  String _message = '';

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        TextField(
          controller: _controller,
          decoration: const InputDecoration(labelText: 'Enter your name'),
        ),
        ElevatedButton(
          onPressed: () => setState(() => _message = 'Hello, ${_controller.text}!'),
          child: const Text('Submit'),
        ),
        Text(_message),
      ],
    );
  }
}

```

Key rules:

- `TextEditingController` reads and controls the text inside a `TextField`.
- Always dispose controllers in `dispose()` to avoid memory leaks (covered fully in Stage 7).

3.5 Scrollable Lists: ListView & GridView

```

ListView.builder(
  itemCount: 50,
  itemBuilder: (context, index) => ListTile(
    leading: const Icon(Icons.person),
    title: Text('User #${index}'),
  ),
);

GridView.count(
  crossAxisCount: 3,
  children: List.generate(9, (i) => Container(
    color: Colors.teal.shade100,
    child: Center(child: Text('$i')),
  )),
);

```

Why `.builder`: `ListView.builder` only constructs the widgets currently visible on screen, which keeps long lists fast and memory-efficient — this matters a lot once you're rendering hundreds or thousands of items from real data in Stage 6.

3.6 Hands-On Project: Profile Card Screen

Build a single Flutter screen that:

1. Shows a circular avatar (`CircleAvatar`), a name, and a short bio, laid out with `Column` and `Row`.
2. Includes a "Follow" `ElevatedButton` that toggles its label between "Follow" and "Following" using

`setState()` .

3. Below the profile, shows a `ListView.builder` of at least 10 "posts" (just placeholder `Card` widgets with text).
4. Uses `Padding` , `SizeBox` , and `Expanded` to make the layout look intentional, not cramped.

This project forces you to combine stateless and stateful widgets, core layout widgets, input handling, and scrollable lists — all of Stage 3.

3.7 Stage 3 Test

Question 1: What is the core difference between `StatelessWidget` and `StatefulWidget` ? When would you choose one over the other?

Question 2: What does calling `setState()` actually do?

Question 3: Explain the difference between `Expanded` and `SizeBox` inside a `Row` .

Question 4: Why is `ListView.builder` preferred over manually creating a `ListView` with a fixed list of children for large datasets?

Question 5: Write a small `Row` containing an `Icon` , an 8-pixel gap, and a `Text` widget.

Passing score: Get all 5 correct before moving to Stage 4.